

Reviewer Pack — Minimal Rank of Configuration Spaces over Boolean Functions ...

Entry baa068d5fc94 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 15:38:35 UTC

Minimal Rank of Configuration Spaces over Boolean Functions vs Decision Tree State Complexity

Entry ID: baa068d5fc94 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 15:38:35 UTC

1. Conjecture

Field A (mathematical branch): Configuration Space Theory **Field B** (complexity object): Complexity Theory: Decision Tree Complexity

Statement:

[‘For a boolean function f , the minimal rank of its configuration space is proportional to the complexity of the smallest decision tree that computes f .’, ‘Formally, for all boolean functions f with n variables, let $R(f)$ be the minimal rank of its configuration space and $T(f)$ be the size of the smallest decision tree computing f . Then, $R(f) = \Theta(T(f))$.’, ‘Additionally, there exists a polynomial-time constructive mapping from each instance of the problem to its corresponding configuration space that can be computed in at most 50 lines of Python code.’]

Rationale (proposer’s reasoning):

[‘The conjecture leverages the geometric properties of configuration spaces to understand the computational complexity of decision trees. Configuration spaces provide a natural way to study the topology of families of sets, which could shed light on the structure of decision trees and their complexity.’, ‘This bridge between geometry and computation may uncover new insights into the hardness of boolean functions and the limitations of decision tree algorithms.’]

Taxonomy category: CONFIG_SPACE_DEC_TREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 09a61bce897fb203

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that the minimal rank of a boolean function’s configuration space is proportional to its decision tree complexity, support is indicated by a Pearson correlation

coefficient (r) ≥ 0.95 between the ranks of configuration spaces and the sizes of corresponding decision trees across all tested functions. Falsification occurs if any seed or instance yields an $r < 0.85$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "configuration space theory" AND "decision tree complexity" AND boolean function" - "minimal rank" config*space* AND decision tree state complexity" - "polynomial-time constructive mapping" config*space* AND decision tree size

Top relevant hits considered: - [http://arxiv.org/abs/1710.07343v1] Fractional Minimal Rank - [http://arxiv.org/abs/2208.01392v2] The Analytic Minimal Rank Sard Conjecture - [http://arxiv.org/abs/2406.15089v2] Sentences over Random Groups II: Sentences of Minimal Rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i + 1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        if A[i][i] == 0:
            continue
        for j in range(n):
            A[i][j] /= A[i][i]
        for k in range(m):
            if k != i and A[k][i] != 0:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]
```

```

def matrix_rank(A):
    rank = 0
    A_copy = [row[:] for row in A]
    gaussian_elimination(A_copy)
    for row in A_copy:
        if any(row):
            rank += 1
    return rank

def decision_tree_size(f, n):
    if n == 1:
        return 1
    var = random.choice(range(n))
    left = [x for x in f if x[var] == 0]
    right = [x for x in f if x[var] == 1]
    return 1 + max(decision_tree_size(left, n - 1), decision_tree_size(right, n - 1))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in range(5, 41):
        f = [tuple(random.randint(0, 1) for _ in range(n)) for _ in range(2**n)]
        R_f = matrix_rank([f[i][j] for j in range(n)] for i in range(2**n))
        T_f = decision_tree_size(f, n)
        results.append((R_f, T_f))
    metric_value = sum(T / R for R, T in results) / len(results)
    instances_tested = len(results)
    conjecture_holds = all(abs(R - T) < 0.1 * max(R, T) for R, T in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "Rank vs Decision Tree Size Ratio",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94693e8f.py", line 78, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94693e8f.py", line 58, in run_trial
    R_f = matrix_rank([[f[i][j] for j in range(n)] for i in range(2*n)])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94693e8f.py", line 39, in matrix_rank
    gaussian_elimination(A_copy)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94693e8f.py", line 23, in gaussian_elim
    if abs(A[j][i]) > abs(A[max_row][i]):
       ~~~~^^^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the Pearson correlation coefficient (r) could not be calculated to determine if it meets the support condition of ≥ 0.95 . | next: Re-run the test with proper error handling and ensure that it produces results for the Pearson correlation coefficient calculation.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13381
2	preregistration	ollama_remote	glm4:latest	0	0	5628
3	novelty	ollama_remote	glm4:latest	0	0	4809
4	novelty	ollama_remote	glm4:latest	0	0	5908
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14419
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7228
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12244
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11027

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	14988

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89631 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/baa068d5fc94.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/baa068d5fc94.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/baa068d5fc94.tar.gz (if generated)